DESENVOLVIMENTO MOBILE

ALEX ZACHARIAS

ALEX@EINSTEINLIMEIRA.COM.BR

RECEBER PARÂMETROS NA CLASSE

- Uma classe pode receber parâmetros ao ser instanciada
- Para chamar essa classe, será obrigatório passar o parâmetro:

```
class Produtos(val cod: Int) {
    var codigo = 0;
    var nome: String = "";
    var quantidade: Int = 0;
    private var preco: Double = 0.0;

    fun setaPreco( p: Double){
        if (p>0) {
            preco = p;
        }
```

```
fun main() {
    val arroz: Produtos = Produtos( cod: 1 )
    val feijao: Produtos = Produtos( cod: 2 )
```

RECEBER PARÂMETROS NA CLASSE

- Podemos utilizar esses parâmetros para carregar um valor inicial nos atributos da classe ou simplesmente utilizar o mesmo como sendo o próprio atributo.

```
class Produtos( val cod: Int) {
    var codigo = cod;
    var nome: String = "";
    var quantidade: Int = 0;
    private var preco: Double = 0.0;
```

- No exemplo acima estamos recebendo o código do produto e colocando o mesmo no atributo código.

CONSTRUTORES

- Métodos construtores são métodos especiais que são chamados automaticamente quando instâncias de uma classe são criadas;
- Os construtores são particularmente úteis para inicializar campos de instâncias de classes para garantir que, quando os métodos dessas classes forem chamados, elas contenham valores específicos e não os valores default;

CONSTRUTORES

- Construtores não retornam nenhum valor;
- Quando a classe for instanciada o construtor será automaticamente chamado;
- Construtores também não podem ser chamados manualmente, são executados apenas quando a instância é criada;
- É possível receber quantidades diferentes de parâmetros fazendo a sobrecarga de construtores.

CONSTRUTORES

- Vamos criar uma classe para calculo da área de um triangulo;
  - Esta classe precisa de 3 atributos (a b c) para os lados do triangulo;
  - Para o calculo da área vamos utilizar a seguinte formula (formula de Heron):

$$area = \sqrt{p(p-a)(p-b)(p-c)} \qquad \text{onde} \qquad p = \frac{a+b+c}{2}$$

CONSTRUTORES

```kotlin
import kotlin.math.sqrt

class Triangulo() {
    var a = 0.0
    var b = 0.0
    var c = 0.0

    fun area(): Double {
        val p = (a + b + c) / 2
        return sqrt( x: p * (p - a) * (p - b) * (p - c))
    }
}
```

CONSTRUTORES

```
fun main() {
    print("Digite o lado A: ")
    val ladoa: String = readln()
    print("Digite o lado B: ")
    val ladob: String = readln()
    print("Digite o lado C: ")
    val ladoc: String = readln()

    val tri:Triangulo = Triangulo()

    tri.a = ladoa.toDouble()
    tri.b = ladoa.toDouble()
    tri.c = ladoa.toDouble()

    println("A área do triangulo é ${tri.area()}")
}
```

CONSTRUTORES

- Agora vamos alterar nossa classe Triangulo criando um construtor para que ela receba os valores dos lados ao ser instanciada:

```
class Triangulo() {
    var a = 0.0
    var b = 0.0
    var c = 0.0

    constructor( ladoa:Double, ladob:Double, ladoc:Double) : this() {
        a = ladoa
        b = ladob
        c = ladoc
    }

    fun area(): Double {
        val p = (a + b + c) / 2.0
```

- O termo this() no final da linha é obrigatório, ele informar que esse construtor descende do construtor padrão

CONSTRUTORES

```
val tri:Triangulo = Triangulo()
tri.a = ladoa.toDouble()
tri.b = ladoa.toDouble()
tri.c = ladoa.toDouble()

val tri2:Triangulo = Triangulo(ladoa.toDouble(), ladob.toDouble(), ladoc.toDouble())
```

- Note que podemos instanciar o objeto utilizando qualquer construtor:
- O triangulo com nome tri esta utilizando o construtor padrão que não recebe parâmetros
- O triangulo tri2 esta utilizando o construtor que criamos que recebe os lados como parâmetros.

CONSTRUTORES - SOBRECARGA

- Podemos criar quantos construtores forem necessários, desde que eles tenham assinaturas diferentes;
- Essa assinatura é composta pelos tipos e quantidades de argumentos que ele recebe;
- Não podemos criar dois construtores com os mesmos argumentos trocando apenas o nome do argumento;
- Esse processo é chamado de sobrecarga e também pode ser utilizado em qualquer método criado.

SOBRECARGA

- Para o método:

  Pessoas(String nome, int idade, double peso, double altura)

- A assinatura é:

  Pessoas(String, int, double, double)

SOBRECARGA

- O tipo de retorno não é considerado como assinatura, portanto não são permitidos dois métodos com o mesmo nome, mesmos tipos de parâmetros, mas tipos de retorno diferentes;

- No caso dos construtores sua assinatura engloba apenas seus parâmetros pois no kotlin o mesmo não tem nome como os métodos.

CONSTRUTORES - SOBRECARGA

- Vamos criar um construtor no nosso triangulo para ser utilizado em triângulos retângulos onde passamos apenas um lado e ele considera que os outros dois lados são iguais:

```
constructor( ladoa:Double ) : this() {
    a = ladoa
    b = ladoa
    c = ladoa
}

constructor( ladoa:Double, ladob:Double, ladoc:Double) : this() {
    a = ladoa
    b = ladob
    c = ladoc
}
```

CONSTRUTORES - SOBRECARGA

- Podemos fazer com que um construtor execute o outro antes de ser executado, passando dentro do this() a assinatura do construtor que queremos executar;
- Isso é útil quando criamos algum código dentro do construtor, assim não precisamos repetir o código dentro do outro construtor.
- No exemplo a seguir, o construtor com 3 parâmetros chama o construtor de um único parâmetro antes de ser executado, e por esse motivo não precisamos carregar o ladoa dentro dele pois já foi carregado no construtor anterior.

CONSTRUTORES - SOBRECARGA

```
constructor( ladoa:Double ) : this() {
    a = ladoa
    b = ladoa
    c = ladoa
}

constructor( ladoa:Double, ladob:Double, ladoc:Double) : this(ladoa) {
    b = ladob
    c = ladoc
}
```

- O fato do primeiro construtor ter carregado o valor do lado A nas variáveis dos lados B e C, não prejudica a execução pois o segundo construtor coloca os valores corretos nessas variáveis

INIT

- Nem sempre conseguimos chamar o outro construtor para poder executar o código que esta nele sem ter que reescrever o mesmo, um exemplo disso é quando um construtor recebe um parâmetro Int e o outro recebe parâmetro de outro tipo;
- Para esses casos podemos utilizar o init { [código] } fazendo com que não seja necessário repetir o código nos diferentes construtores.
- Podemos também utilizar o init sem ter criado nenhum construtor.
- O init é um código que será executado automaticamente quando o objeto for instanciado.

INIT

```
class Triangulo() {
    var a = 0.0
    var b = 0.0
    var c = 0.0

    init {
        println( "O objeto foi instanciado!")
    }

    constructor( ladoa:Double ) : this() {
        a = ladoa
```

- No exemplo acima estamos escrevendo a frase O objeto foi instanciado! toda vez que instanciamos a classe Triangulo.

GETTER AND SETTER

- Na maioria das linguagens orientadas a objetos quando precisamos bloquear ou controlar (fazer testes ou validações nos dados) as alterações de um atributo de fora da classe, criamos esse atributo como privado e criamos dois métodos: um para alterar o valor e outro para mostrar o mesmo.

- O Kotlin faz isso de uma forma mais implícita tentando facilitar para o usuário.

GETTER AND SETTER

- No Kotlin quando não queremos que um atributo seja alterado ou acessado de fora da classe em momento nenhum devemos criar o mesmo como private.
- O mesmo vale para métodos.
- Agora, se queremos controlar o retorno do valor lido, ou o valor que vai ser gravado, criamos o atributo normalmente, e informamos abaixo dele seu get() e seu set(), podendo usar ambos ou apenas um conforme necessário.

GETTER AND SETTER

- Exemplo em Java:

```java
public class Pessoa {
    no usages
    String nome;
    2 usages
    private double peso;
    no usages
    public void setPeso( double peso){
        if (peso > 0 && peso < 500) {
            this.peso = peso;
        }
    }
    no usages
    public double getPeso(){
        return peso;
    }
}
```

- Mesmo exemplo em Kotlin:

```kotlin
class Pessoa {
    var nome:String = ""
    var peso:Double = 0.0
        set(value) {
            if (value > 0 && value < 500){
                field = value
            }
        }
        get(){
            return field
        }
}
```

GETTER AND SETTER

- Dentro do get() ou do set() em Kotlin devemos referenciar o atributo em questão utilizando o termo field, porque se utilizarmos no nome do atributo ele vai executar o get() ou o set() novamente fazendo com que o programa fique em looping.

- Como não fizemos nenhum tratamento de dados dentro do get(), podemos omitir o mesmo:

```
class Pessoa {
    var nome:String = ""
    var peso:Double = 0.0
        set(value) {
            if (value > 0 && value < 500){
                field = value
            }
        }
}
```

GETTER AND SETTER

- Caso precise bloquear apenas as alterações no atributo permitindo apenas de dentro da classe utilize um private set.

```
class Pessoa {
    var nome:String = ""
    var peso:Double = 0.0
        set(value) {
            if (value > 0 && value < 500){
                field = value
            }
        }
    var imc: double = 0.0
        private set
}
```

- O atributo imc só pode ser alterado de dentro da classe, mas pode ser acessado externamente.

HERANÇA

- Herança é o ato de criar uma classe a partir de outras previamente criadas.
- Essa nova classe é chamada de subclasse ou classe derivada.
- A classe que deu origem a uma subclasse é chamada de superclasse ou classe base.
- Uma subclasse herda métodos e atributos de sua superclasse, e permite que esses métodos sejam reescritos de acordo com a necessidade dessa subclasse.

HERANÇA

- Imagine a seguinte situação:
  - A Einstein tem dois tipos de funcionários: Administrativos e Professores
  - Que podem ser representados pelas classes:

Professores
nome: String matricula: Int curso: String salario: Double
setMatricula(Int) setNome(String) setSalario(Double) setCurso(String) getMatricula(): Int getNome(): String getSalario(): Double getCurso(): String aumentoSalario(Double)

Administrativos
nome: String matricula: Int departamento: String salario: Double
setMatricula(Int) setNome(String) setSalario(Double) setDepartamento(String) getMatricula(): Int getNome(): String getSalario(): Double getDepartamento(): String aumentoSalario(Double)

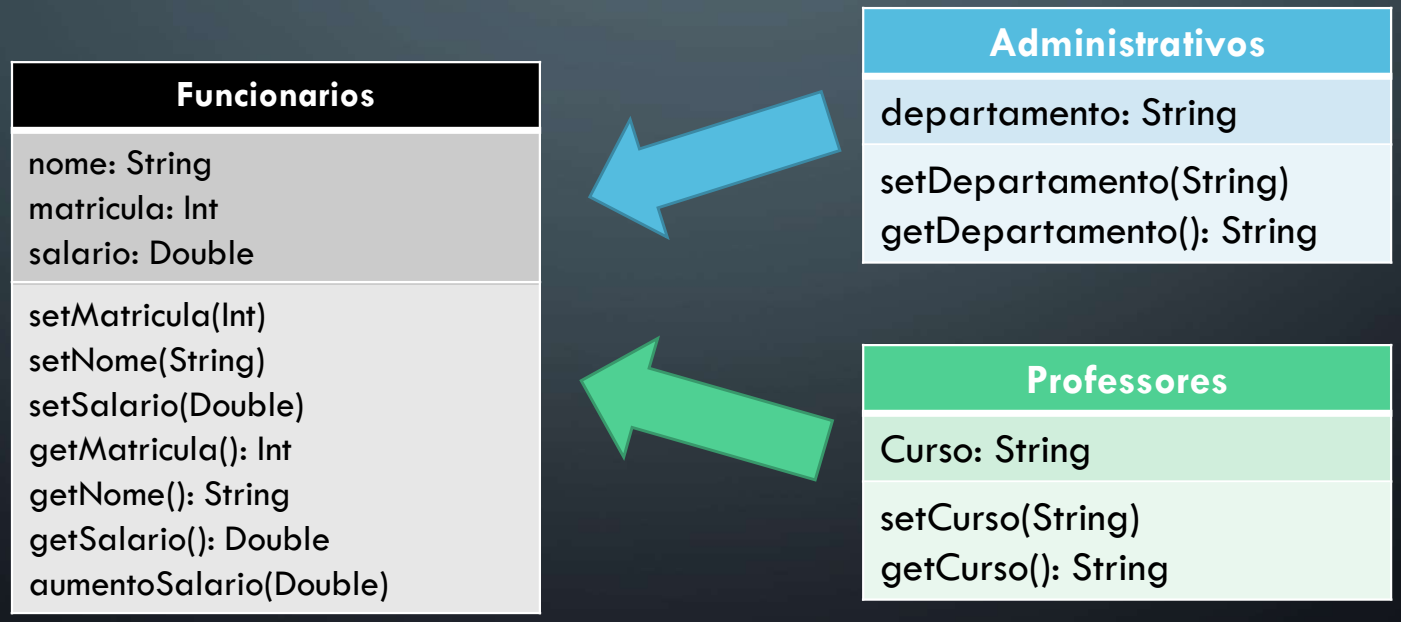
HERANÇA

- Essas duas classes tem muitas coisas em comum;
- A única diferença entre essas duas classes é o atributo curso/departamento e seus respectivos get e set.

Professores	Administrativos
nome: String	nome: String
matricula: Int	matricula: Int
curso: String	departamento: String
salario: Double	salario: Double
setMatricula(Int)	setMatricula(Int)
setNome(String)	setNome(String)
setSalario(Double)	setSalario(Double)
setCurso(String)	setDepartamento(String)
getMatricula(): Int	getMatricula(): Int
getNome(): String	getNome(): String
getSalario(): Double	getSalario(): Double
getCurso(): String	getDepartamento(): String
aumentoSalario(Double)	aumentoSalario(Double)

HERANÇA

- Podemos criar uma classe base que terá os atributos e métodos comuns a essas duas classes, e criar cada uma delas **herdando** essa classe e incluindo apenas o que é especifico de cada uma delas.
- Esse processo é chamado de **Herança.**



HERANÇA

- Note que a classe Funcionários é a classe base para as classes Administrativos e Professores;
- Falamos que Administrativos e Professores **estendem** a classe Funcionários, ou seja, Administrativos é um tipo de Funcionário por exemplo;
- Sendo assim Administrativos e Professores possuem os mesmos atributos e métodos da classe Funcionários;
- Vamos criar o código:

HERANÇA

- Para que uma classe possa ser herdada no kotlin precisamos utilizar o operador open na declaração dela.
- Só precisamos criar o método get de um atributo se for necessário alterar seu código padrão, caso contrario o kotlin faz automaticamente.

```
open class Funcionario {
    var matricula: Int = 0
        set(value) {
            if (value>0) field = value
        }
    var nome: String = ""
        set(value){
            if (value != "") field = value
        }
    var salario: Double = 0.0
        set(value){
            if (value>0) field = value
        }
    fun aumentoSalario( porcentagem: Double){
        if (porcentagem > 0) {
            val valor = salario * porcentagem / 100
            salario += valor
        }
    }
}
```

HERANÇA

```
class Administrativo: Funcionario() {
    var departamento: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Departamento não cadastrado"
        }
}

class Professor: Funcionario() {
    var curso: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Curso não cadastrado"
        }
}
```

HERANÇA

- Tanto a classe Professor quanto a classe Administrativo estão herdando a classe Funcionario
- Essas classes derivadas não precisam do operador open, ele só é necessário na superclasse (classe que vai ser herdada)
- Criamos um método get para retornar um texto quando o departamento ou o curso não forem informados.

HERANÇA

- O que fizemos:
  - Criamos uma classe base Funcionario;
  - Criamos uma classe Professores que estende (herda) a classe Funcionario;
  - Criamos uma classe Administrativos que estende (herda) a classe Funcionario;
  - As classes Administrativos e Professores possuem seus atributos e métodos exclusivos mais os atributos e métodos da classe Funcionario;
- Herança permite economia de código.

Cadastro.kt

```kotlin
fun main() {
    print("Digite a matricula: ")
    var entrada = readln()
    val matricula = entrada.toInt()
    print("Digite o nome: ")
    val nome = readln()
    print("Digite o salário: ")
    entrada = readln()
    val salario = entrada.toDouble()
    print("Digite o curso: ")
    val curso = readln()

    val prof1 = Professor()
    prof1.matricula = matricula
    prof1.nome = nome
    prof1.salario = salario
    prof1.curso = curso

    println("Matricula: ${prof1.matricula}")
    println("Nome.....: ${prof1.nome}")
    println("Salario..: ${prof1.salario}")
    println("Curso....: ${prof1.curso}")
}
```

```
Digite a matricula: 15
Digite o nome: Jose da Silva
Digite o salário: 2500.35
Digite o curso: Engenharia
Matricula: 15
Nome.....: Jose da Silva
Salario..: 2500.35
Curso....: Engenharia

Process finished with exit code 0
```

HERANÇA

- O que fizemos:
  - Instanciamos apenas a classe Professores;
  - Os atributos **nome, matricula e salario** são da classe Funcionário que foi herdada por Professores;
  - O atributo **curso** é exclusivos da classe Professores;

- Vamos criar um método construtor para a classe Funcionario para receber os atributos diretamente ao instanciar:

HERANÇA

- Note que para podermos criar um construtor precisamos colocar () na declaração da classe (1ª linha do código)
- Como criamos uma variável nome dentro do construtor com o mesmo nome do atributo, precisamos utilizar o comando this. para identificar o atributo.

```
open class Funcionario() {
    var matricula: Int = 0
        set(value) {
            if (value>0) field = value
        }
    var nome: String = ""
        set(value){
            if (value != "") field = value
        }
    var salario: Double = 0.0
        set(value){
            if (value>0) field = value
        }

    constructor(mat:Int, nome:String, sal:Double): this(){
        matricula = mat
        this.nome = nome
        salario = sal
    }

    fun aumentoSalario( porcentagem: Double){
        if (porcentagem > 0) {
            val valor = salario * porcentagem / 100
            salario += valor
        }
    }
}
```

HERANÇA

```
class Professor: Funcionario {
    var curso: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Curso não cadastrado"
        }

    constructor(mat:Int, nome:String, sal:Double, curso: String): super(mat, nome, sal) {
        this.curso = curso
    }
}
```

- Na classe Professor criamos um construtor para atualizar o atributo curso e configuramos ele para chamar o construtor da super classe Funcionario para atualizar os outros três atributos.
- Note que na declaração da classe não podemos utilizar () porque isso força ele a chamar a classe Funcionario utilizando o construtor padrão.

HERANÇA

- Vamos fazer o mesmo na classe Administrativo:

```
class Administrativo: Funcionario {
    var departamento: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Departamento não cadastrado"
        }
    constructor(mat:Int, nome:String, sal:Double, departamento: String): super(mat, nome, sal) {
        this.departamento = departamento
    }
}
```

HERANÇA

- Agora podemos simplificar o código do cadastro, passando os valores dos atributos ao instanciar o objeto:

```
fun main() {
    print("Digite a matricula: ")
    var entrada = readln()
    val matricula = entrada.toInt()
    print("Digite o nome: ")
    val nome = readln()
    print("Digite o salário: ")
    entrada = readln()
    val salario = entrada.toDouble()
    print("Digite o curso: ")
    val curso = readln()

    val prof1 = Professor( matricula, nome, salario, curso )

    println("Matricula: ${prof1.matricula}")
    println("Nome.....: ${prof1.nome}")
    println("Salario..: ${prof1.salario}")
    println("Curso....: ${prof1.curso}")
}
```

OVERRIDE - SOBREESCRITA

- Quando uma classe possui um método que ela herdou e desejamos alterar esse método podemos utilizar o comando override para sobrescrever o mesmo.
- Vamos utilizar como exemplo o método toString(), toda classe possui esse método, ele retorna um texto com o conteúdo dessa classe.
- Utilizamos ele quando damos um println em uma variável numérica, o toString() retorna o valor da variável como um texto.

OVERRIDE - SOBREESCRITA

- Se utilizarmos o comando abaixo, ele vai retornar um texto contendo o nome da classe desse objeto, seguido de @ e um endereço:

```
println(prof1)
```

- Retorno:

```
Professor@70177ecd
```

OVERRIDE - SOBREESCRITA

- Podemos sobrescrever o método **toString()** para que ele retorne um texto mais amigável para o usuário:

```
class Professor: Funcionario {
    var curso: String = ""
        set(value) {
            if(value != "") field = value
        }
        get() {
            return if (field != "") field else "Curso não cadastrado"
        }

    constructor(mat:Int, nome:String, sal:Double, curso: String): super(mat, nome, sal) {
        this.curso = curso

    }

    override fun toString(): String {
        return "matricula: $matricula" +
                "\nNome.....: $nome" +
                "\nSalario..: $salario" +
                "\nCurso....: $curso"
    }
}
```

OVERRIDE - SOBREESCRITA

- Agora podemos simplificar mais ainda o programa:

```
fun main() {
    print("Digite a matricula: ")
    var entrada = readln()
    val matricula = entrada.toInt()
    print("Digite o nome: ")
    val nome = readln()
    print("Digite o salário: ")
    entrada = readln()
    val salario = entrada.toDouble()
    print("Digite o curso: ")
    val curso = readln()

    val prof1 = Professor( matricula, nome, salario, curso )

    println(prof1)
}
```